NAWATECH TECHNICAL DOCUMENT

Name : **Muhammad Rifqi**
Url Project : Github = https://github.com/muhammad-rifqi/nawatech [public repository]

1. Entity Relational Diagram

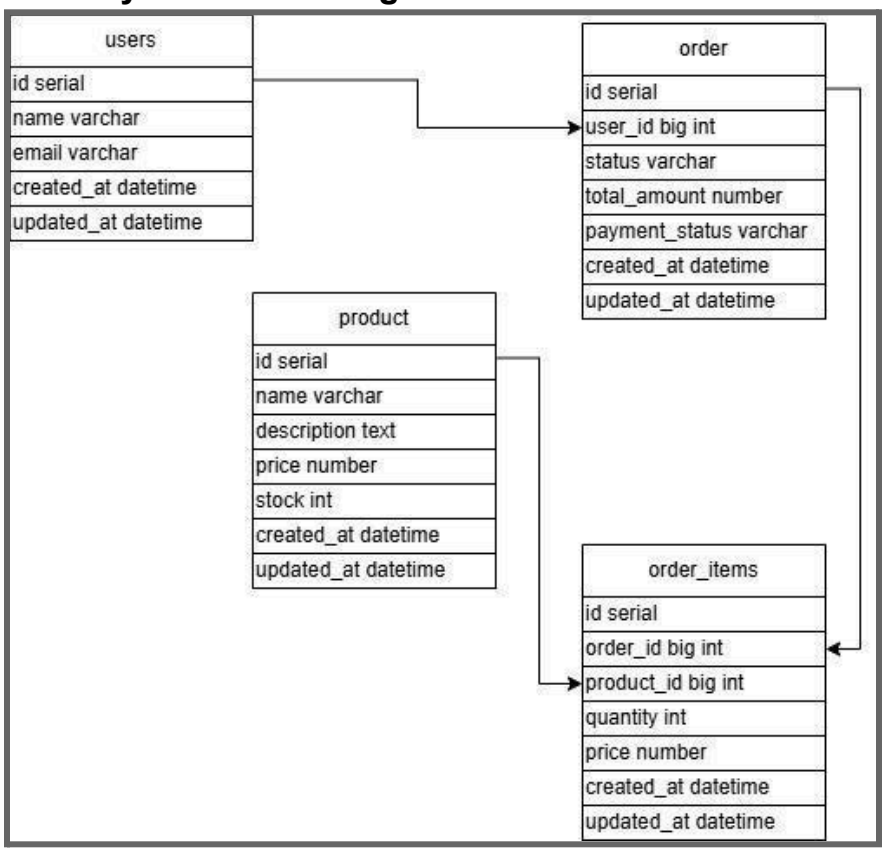

Figure 1.0

From the image above, we can see that an actor named "users" with attributes [id, name, email] has a relationship with the "order" table, with a field named user_id, and a one-to-many relationship. This means that one user can have many orders.

Furthermore, the order table has a one-to-many relationship with the order_item table, meaning that one order can have many order_items.

The order_item table also has a one-to-many relationship with the product table. This means that one user can have many orders, and each order contains many items, including attributes of the product itself.

2. Application Demo

a. Technology used

Laravel 10.*	Postman	PostgreSQL	Redis	PHP	K6

b. Architecture Laravel

In the application that I developed, I used a pattern called *"Service Layer Architecture with Repository Pattern (Layered Architecture)."* Here I use

Controller
↓
Service (Business Logic)
↓
Repository (Data Access / Query DB)
↓
Model / Database

in my application code I use as follows :

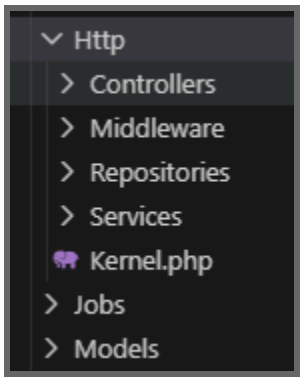

Figure 1.1

C. design and optimize the existing database schema.

(1) , Add the index in the field of table in database, I use *"CREATE INDEX idx_users_email ON users (email);"* for query sample on the users table

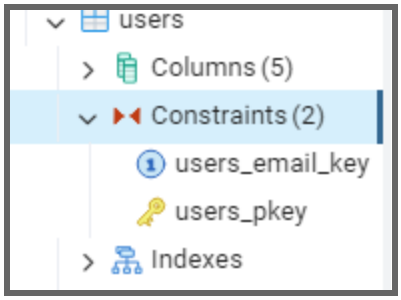

Figure 1.2

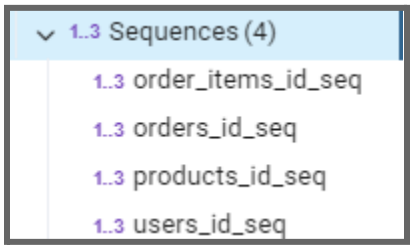

Figure 1.3

then I use **(2), redis** for database cache and also use the built-in **(3), cache from laravel**. Like this *"use Illuminate\Support\Facades\Cache;" and "composer require predis/predis"*

for the example cache use redis i use cache dan redis with parallels in the same time :

```
return Cache::remember("orders_limit_{$limit}", 600, function () use ($limit) {

return Orders::with([
'user:id,name',
'items:id,order_id,product_id',
'items.product:id,name'
])
->select('id','user_id', 'payment_status')
->orderBy('id', 'desc')
->limit($limit)
->get()
->toArray();

});
```

D. Build an Endpoint to Retrieve All Orders

The following is a list of application programming interfaces(api) that have been created.

Order List

Method : GET

URL : http://localhost:8000/api/orders

Response : JSON

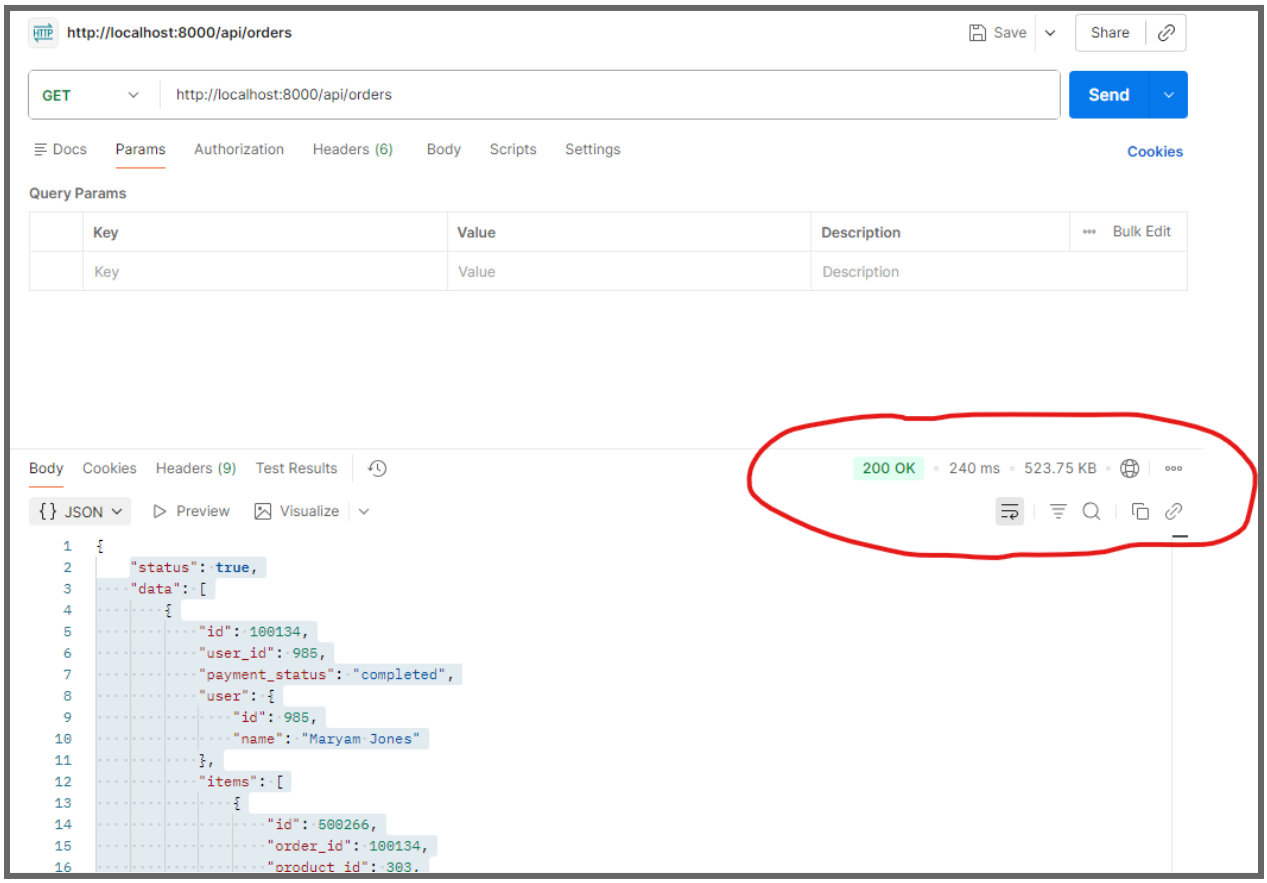

Figure 1.4

From the image above, it can be seen that the response time is around 240 ms for the second attempt, but for the first attempt it can reach 5s for thousands of data.

for the display on redis like this

```
127.0.0.1:6379[1]> keys *
1) "laravel_database_laravel_cache_:orders_limit_1000"
2) "laravel_database_laravel_cache_:a75f3f172bfb296f2e10cbfc6dfc1883"
3) "laravel_database_laravel_cache_:a75f3f172bfb296f2e10cbfc6dfc1883:timer"
127.0.0.1:6379[1]>
```

Figure 1.5

With k6 no cache

```
        /\      Grafana   /‾‾/
   /\  /  \     |\  __   /  /
  /  \/    \    | |/ /  /   ‾‾\
 /          \   |   (  |  (‾)  |
/ __________ \  |_|\_\  \_____/


    execution: local
       script: script.js
       output: -

    scenarios: (100.00%) 1 scenario, 50 max VUs, 1m30s max duration (incl. graceful stop):
             * default: 50 looping VUs for 1m0s (gracefulStop: 30s)


 TOTAL RESULTS

   HTTP
   http_req_duration....: avg=17.68s min=3.52s med=19.23s max=23.29s p(90)=19.34s p(95)=19.55s
   http_req_failed......: 100.00% 187 out of 187
   http_reqs............: 187     2.34131/s

   EXECUTION
   iteration_duration...: avg=18.68s min=4.52s med=20.23s max=24.29s p(90)=20.34s p(95)=20.55s
   iterations...........: 187     2.34131/s
   vus..................: 3       min=3          max=50
   vus_max..............: 50      min=50         max=50

   NETWORK
   data_received........: 196 MB  2.4 MB/s
   data_sent............: 65 kB   810 B/s
```

Figure 1.6

For the class model orderItem

```php
class Orderitems extends Model
{
    use HasFactory;
    protected $table = 'order_items';
     protected $fillable = [
        'order_id',
        'product_id',
        'quantity' ,
        'price'
    ];

    public function product()
    {
        return $this->belongsTo(Product::class);
    }

    public function order()
    {
        return $this->belongsTo(Order::class, 'order_id');
    }

}
```

Figure 1.7

For the class model Order

```php
class Orders extends Model
{
    use HasFactory;
    protected $table = 'orders';
    protected $fillable = ['user_id'];

    public function user()
    {
        return $this->belongsTo(Users::class, 'user_id', 'id');
    }

    public function items()
    {
        return $this->hasMany(OrderItems::class, 'order_id');
    }
}
```

Figure 1.8

For the class model users

```php
class Users extends Model
{
    use HasFactory;
    protected $table = 'users';

     public function order()
    {
        return $this->belongsTo(Order::class, 'user_id');
    }
}
```

Figure 1.9

E. Implement Order Creation with Overselling Prevention

Order Store Multiple user

Method : POST

URL : http://localhost:8000/api/store

Payload : [

```
{
  "user_id": 215,
  "status": "pending",
  "items": [
    {"product_id": 301, "quantity": 2, "price": 50000},
    {"product_id": 302, "quantity": 1, "price": 25000}
  ]
},
{
  "user_id": 218,
  "items": [
    {"product_id": 303, "quantity": 1, "price": 75000}
  ]
}
]
```

Response : JSON

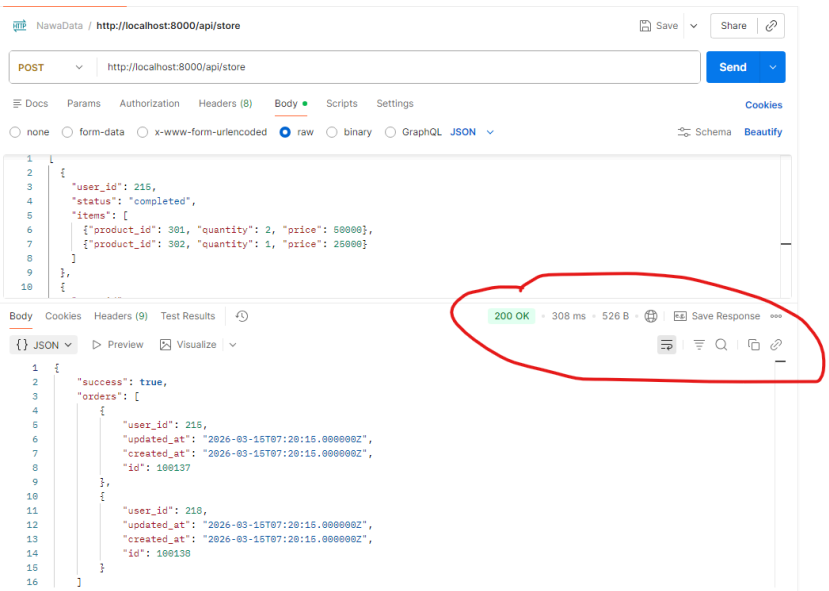

Figure 2.0

From the image above, it can be seen that the response time is around 308 ms for the second attempt, but for the first attempt it can reach 2s for ***multiple insert data.***

for the display on k6 like this, for *testing 500 users* accessing simultaneously *within a period of 10 minutes.*

```
D:\web_app\laravel\nawatechapp\nawatech\k6>k6

        /\      Grafana   /‾‾/
    /\  /  \     |\  __   /  /
   /  \/    \    | |/ /  /   ‾‾\
  /          \   |   (  |  (‾)  |
 / ‾‾‾‾‾‾‾‾‾‾ \  |_|\_\  \_____/

Grafana k6 is an easy-to-use, open-source load and performance testing tool

Usage:
  k6.exe [command]

Core Commands:
  new         Create a test
  run         Run a test
  cloud       Run and manage Grafana Cloud tests

Additional Commands:
  archive     Create an archive
  completion  Generate the autocompletion script for the specified shell
  deps        Resolve and list the dependencies of a test
  inspect     Inspect a script or archive

Flags:
  -h, --help      Show help
      --version   Show version information

Examples:
  # Create a test
  $ k6.exe new test.js

  # Run a test
  $ k6.exe run test.js

  # Run a test in Grafana Cloud
  $ k6.exe cloud run test.js

  # Run locally, stream results to Grafana Cloud
  $ k6.exe cloud run --local-execution test.js

Use "k6.exe [command] --help" for more information about a command.
Full CLI documentation: https://grafana.com/docs/k6/v1.6.x/
```

Figure 2.1

```
TOTAL RESULTS

  HTTP
  http_req_duration..............: avg=587.48ms min=0s       med=0s     max=59.97s p(90)=0s     p(95)=0s
    { expected_response:true }...: avg=37.47s   min=332.27ms med=39.05s max=58.14s p(90)=47.22s p(95)=50.13s
  http_req_failed................: 99.65% 188331 out of 188991
  http_reqs......................: 188991 299.981865/s

  EXECUTION
  iteration_duration.............: avg=1.58s    min=1s       med=1s     max=1m1s   p(90)=1s     p(95)=1s
  iterations.....................: 188987 299.975515/s
  vus............................: 115    min=115              max=500
  vus_max........................: 500    min=500              max=500

  NETWORK
  data_received..................: 15 MB  24 kB/s
  data_sent......................: 1.0 MB 1.6 kB/s
```

Figure 2.2

It appears that the average is around 587 ms, which has not yet reached the desired target.

F. Implement 'Dummy' Async Payment Processing

Order payment API

Method : POST

URL : http://localhost:8000/api/orders/101461/payment

Payload : { }

Response : JSON

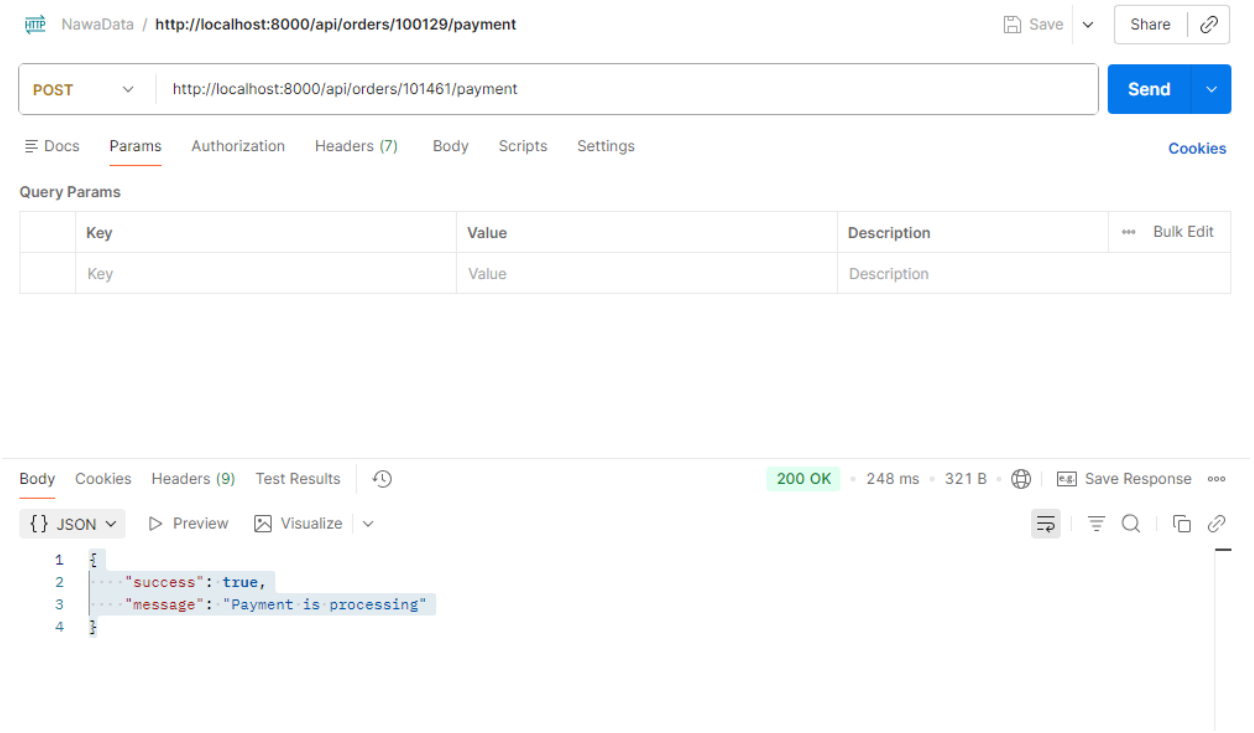

Figure 2.3

Previous database :

101455	101463	311	pending	125000.00	pending	2026-03-15 08:23:48	2026-03-15 08:23:48
101456	101464	333	pending	75000.00	pending	2026-03-15 08:23:48	2026-03-15 08:23:48

Figure 2.4

if we run the application programming interface (API) above

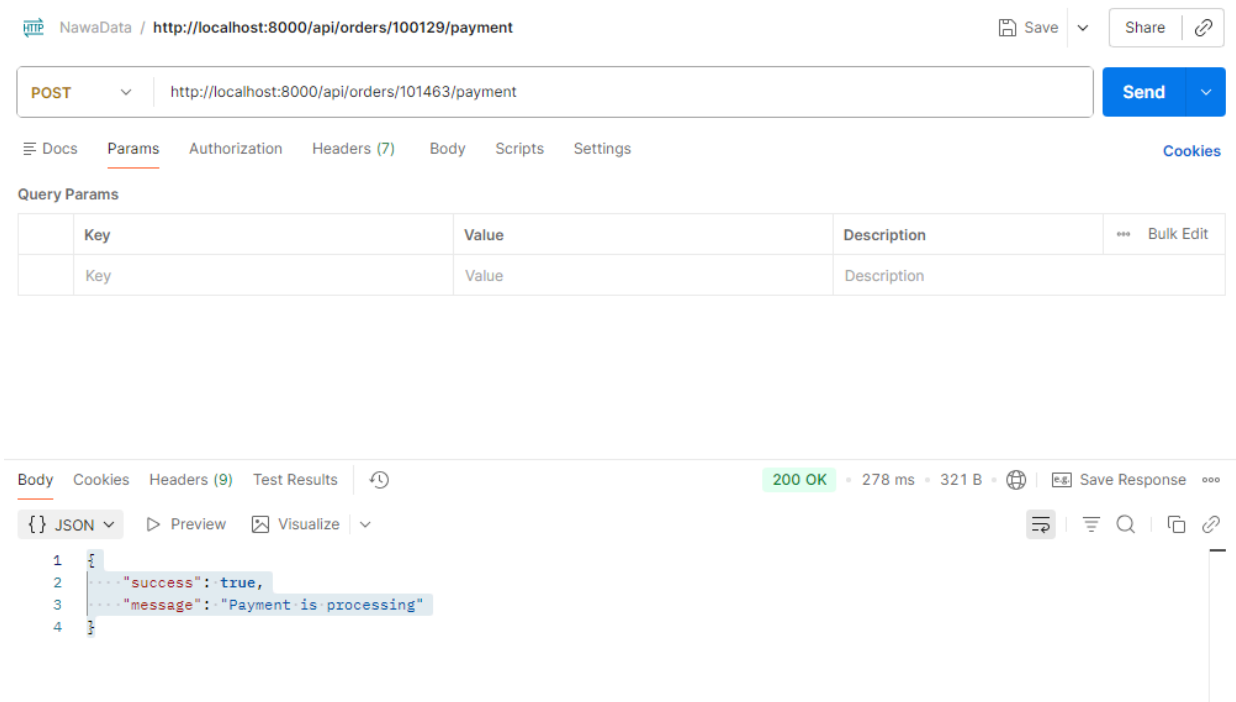

Figure 2.5

The output :

101455	101463	311	completed	125000.00	paid	2026-03-15 08:23:48	2026-03-15 08:27:50

Figure 2.6

Jobs

```
D:\web_app\laravel\nawatechapp\nawatech>php artisan queue:work redis
  2026-03-15 08:11:41 App\Jobs\ProcessPaymentJob ..... RUNNING
  2026-03-15 08:11:43 App\Jobs\ProcessPaymentJob ..... 2s DONE
  2026-03-15 08:13:01 App\Jobs\ProcessPaymentJob ..... RUNNING
  2026-03-15 08:13:03 App\Jobs\ProcessPaymentJob ..... 2s DONE
  2026-03-15 08:27:48 App\Jobs\ProcessPaymentJob ..... RUNNING
  2026-03-15 08:27:50 App\Jobs\ProcessPaymentJob ..... 2s DONE
```

```
    protected $orderId;

    public function __construct($orderId)
    {
        $this->orderId = $orderId;
    }

    /**
     * Execute the job.
     */
    public function handle(OrderService $orderService): void
    {
        //
        sleep(2);
        $status = rand(0,1) ? 'paid' : 'failed';
        $orderService->updatePaymentStatus($this->orderId, $status);
    }
```

Figure 2.7

G. Build Report Endpoint with Caching

Report Order API

Method : GET

URL : http://localhost:8000/api/report

Response : JSON

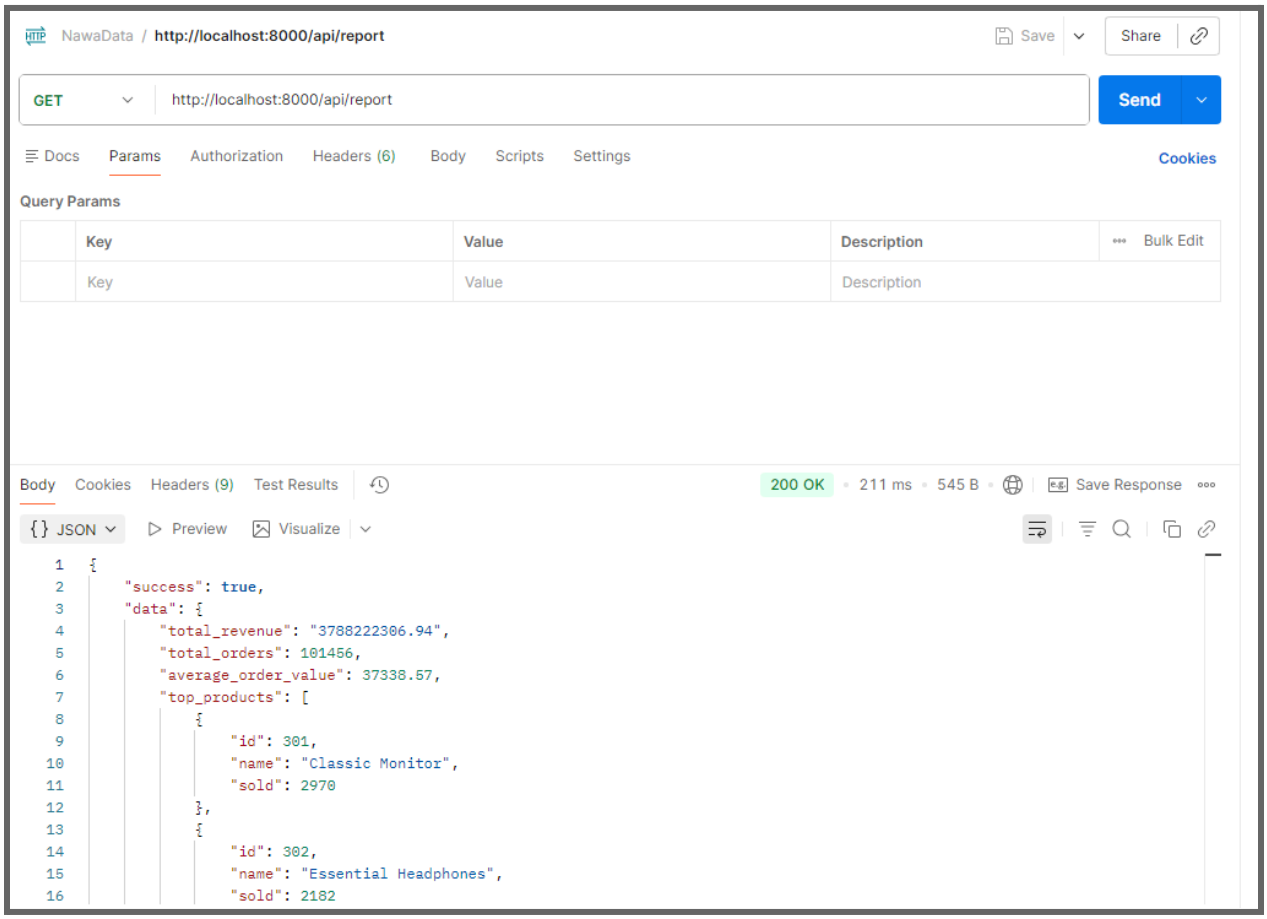

Figure 2.8

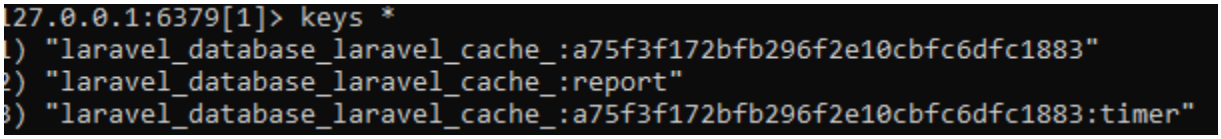

Figure 2.9

H. List All Route API

```
Route::get('/users', [App\Http\Controllers\UsersController::class, 'index']);
Route::get('/orders', [App\Http\Controllers\OrdersController::class, 'index']);
Route::post('/store', [App\Http\Controllers\OrdersController::class, 'store']);
Route::get('/orders_detail/{id}', [App\Http\Controllers\OrdersController::class, 'show']);
Route::post('/orders/{id}/payment', [App\Http\Controllers\OrdersController::class, 'update']);
Route::get('/report', [App\Http\Controllers\OrdersController::class, 'report']);
```

Figure 3.0